SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.* (BL2, BL3)

Activity Tool : Case Study (Medium)
Assessment Tool Weightage: 30%

QUESTIONS Total Marks: 50

Q.No	Question	Bloom's Level	Marks
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	**5**
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	**5**
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	**5**
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	**5**
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	**5**
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	**5**

7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	**5**
8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	**5**
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	**5**
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	**5**

Code of Conduct:

I Lokesh S (192511037) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning

Solution Design	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q1. Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.

Answer

Given relation:

ORDER(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

Step 1: Identify Functional Dependencies

Assume each customer has one name and each product has one name and price.

CustID -> CustName

ProdID -> ProdName, Price

(OrderID, ProdID) -> Qty

If an order contains multiple products, the combination of OrderID and ProdID identifies each order item.

Therefore:

Candidate Key = (OrderID, ProdID)

Step 2: Identify Partial Dependencies

The candidate key is:

(OrderID, ProdID)

But:

ProdID -> ProdName, Price

Here, ProdName and Price depend only on part of the composite key, ProdID.

Similarly, customer information depends on CustID.

This creates redundancy.

Step 3: Decompose into 2NF

Create CUSTOMER:

CUSTOMER(CustID, CustName)

Create PRODUCT:

PRODUCT(ProdID, ProdName, Price)

Create ORDER:

ORDER(OrderID, CustID)

Create ORDER_ITEM:

ORDER_ITEM(OrderID, ProdID, Qty)

Step 4: Check 3NF

The resulting relations have no transitive dependency.

Final 3NF Design

CUSTOMER(CustID, CustName)

PRODUCT(ProdID, ProdName, Price)

ORDER(OrderID, CustID)

ORDER_ITEM(OrderID, ProdID, Qty)

Advantages

- Removes customer information redundancy.
- Removes product information redundancy.
- Eliminates partial dependencies.
- Prevents update anomalies.
- Maintains data integrity.

Q2. A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.

Answer

Given relation:

PATIENT(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

Assume the following functional dependencies:

PID -> PName, DocID, WardNo

DocID -> DocName, Specialization

WardNo -> WardName

Step 1: Identify Candidate Key

PID uniquely identifies a patient.

Candidate Key = PID

Step 2: Identify Transitive Dependencies

We have:

PID -> DocID

DocID -> DocName, Specialization

Therefore:

PID -> DocName, Specialization

Similarly:

PID -> WardNo

WardNo -> WardName

Therefore:

PID -> WardName

These are transitive dependencies.

Step 3: Identify Anomalies

Update anomaly: Doctor information is repeated for every patient treated by the doctor.

Insertion anomaly: A new doctor or ward cannot be stored independently without patient information.

Deletion anomaly: Deleting the last patient associated with a doctor may also delete the doctor's information.

Step 4: BCNF Decomposition

Create PATIENT:

PATIENT(PID, PName, DocID, WardNo)

Create DOCTOR:

DOCTOR(DocID, DocName, Specialization)

Create WARD:

WARD(WardNo, WardName)

Final BCNF Design

PATIENT(PID, PName, DocID, WardNo)

DOCTOR(DocID, DocName, Specialization)

WARD(WardNo, WardName)

In each relation, the determinant of every non-trivial FD is a key.

Therefore, the design is in BCNF.

Q3. Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.

Answer

Given:

ENROLLMENT(SID, SName, CourseID, CourseName, Faculty, Grade)

Step 1: Identify Functional Dependencies

A student ID determines the student name:

SID -> SName

A course ID determines the course name and faculty:

CourseID -> CourseName, Faculty

A student's grade depends on the student and course:

(SID, CourseID) -> Grade

Therefore, the candidate key is:

Candidate Key = (SID, CourseID)

Step 2: Identify Partial Dependencies

The candidate key contains two attributes:

(SID, CourseID)

But:

SID -> SName

SName depends only on SID.

Also:

CourseID -> CourseName, Faculty

CourseName and Faculty depend only on CourseID.

Therefore, partial dependencies exist.

This violates 2NF.

Step 3: Decompose

Create STUDENT:

STUDENT(SID, SName)

Create COURSE:

COURSE(CourseID, CourseName, Faculty)

Create ENROLLMENT:

ENROLLMENT(SID, CourseID, Grade)

Final 2NF Design

STUDENT(SID, SName)

COURSE(CourseID, CourseName, Faculty)

ENROLLMENT(SID, CourseID, Grade)

Result

Partial dependencies are removed, redundancy is reduced, and the relations are in 2NF.

Q4. Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.

Answer

Given:

BOOKING(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)

Assume the following functional dependencies:

PassID -> PassName

FlightNo -> DeptCity, ArrCity, Date

(FlightNo, PassID) -> Seat

Step 1: Identify Candidate Key

A passenger can book a particular flight.

Therefore:

Candidate Key = (FlightNo, PassID)

Step 2: Check 2NF

We have:

PassID -> PassName

and:

FlightNo -> DeptCity, ArrCity, Date

These are partial dependencies because they depend on only part of the composite key.

Therefore, decompose.

Step 3: 2NF Decomposition

Create PASSENGER:

PASSENGER(PassID, PassName)

Create FLIGHT:

FLIGHT(FlightNo, DeptCity, ArrCity, Date)

Create BOOKING:

BOOKING(FlightNo, PassID, Seat)

Step 4: Check BCNF

In PASSENGER:

PassID -> PassName

PassID is the key.

In FLIGHT:

FlightNo -> DeptCity, ArrCity, Date

FlightNo is the key.

In BOOKING:

(FlightNo, PassID) -> Seat

(FlightNo, PassID) is the key.

Therefore, every determinant is a superkey.

Final BCNF Design

PASSENGER(PassID, PassName)

FLIGHT(FlightNo, DeptCity, ArrCity, Date)

BOOKING(FlightNo, PassID, Seat)

Thus, the airline booking schema is normalized up to BCNF.

Q5. Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.

Answer

Consider:

EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID, ManagerName)

Step 1: Identify Functional Dependencies

EmpID -> EmpName, DeptID

DeptID -> DeptName, ManagerID

ManagerID -> ManagerName

Step 2: Identify Transitive Dependencies

From:

EmpID -> DeptID

DeptID -> DeptName

we get:

EmpID -> DeptName

Also:

EmpID -> DeptID

DeptID -> ManagerID

ManagerID -> ManagerName

Therefore:

EmpID -> ManagerName

These are transitive dependencies.

Step 3: Normalize to 3NF

Create EMPLOYEE:

EMPLOYEE(EmpID, EmpName, DeptID)

Create DEPARTMENT:

DEPARTMENT(DeptID, DeptName, ManagerID)

Create MANAGER:

MANAGER(ManagerID, ManagerName)

Final 3NF Design

EMPLOYEE(EmpID, EmpName, DeptID)

DEPARTMENT(DeptID, DeptName, ManagerID)

MANAGER(ManagerID, ManagerName)

Result

The transitive dependencies are removed, redundancy is reduced, and data integrity is improved.

Q6. Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.

Answer

Given:

LIBRARY(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)

For this case, assume a member can borrow many books and that a book can have multiple authors independently.

Step 1: Functional Dependencies

MemberID -> MemberName

BookID -> Title, Genre

If author information is independently associated with a book:

BookID ->-> Author

This is a multivalued dependency.

Step 2: Identify MVD

The important MVD is:

BookID ->-> Author

This means a book can have multiple authors independently of other information.

Step 3: 4NF Violation

A relation is in 4NF when every non-trivial MVD has a determinant that is a superkey.

If BookID is not a superkey of the entire LIBRARY relation, the MVD causes a 4NF violation.

Step 4: Decompose

Create BOOK:

BOOK(BookID, Title, Genre)

Create BOOK_AUTHOR:

BOOK_AUTHOR(BookID, Author)

Create MEMBER:

MEMBER(MemberID, MemberName)

Create BORROW:

BORROW(MemberID, BookID, BorrowDate)

Final 4NF Design

MEMBER(MemberID, MemberName)

BOOK(BookID, Title, Genre)

BOOK_AUTHOR(BookID, Author)

BORROW(MemberID, BookID, BorrowDate)

This separates independent multivalued information and reduces redundancy.

Q7. Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.

Answer

Consider:

BILLING(CustomerID, CustomerName, PhoneNo, PlanID, PlanName, Amount)

Step 1: Functional Dependencies

Customer ID identifies customer information:

CustomerID -> CustomerName, PhoneNo

Plan ID identifies plan information:

PlanID -> PlanName, Amount

If phone number is unique:

PhoneNo -> CustomerID

Step 2: Candidate Keys

From:

CustomerID -> CustomerName, PhoneNo

CustomerID can identify the customer.

Therefore:

Candidate Key = CustomerID

If PhoneNo uniquely identifies a customer:

PhoneNo -> CustomerID

then:

Candidate Key = PhoneNo

Thus possible candidate keys are:

CustomerID

PhoneNo

Step 3: Select Primary Key

Choose one candidate key as the primary key.

For example:

Primary Key = CustomerID

PhoneNo becomes an alternate/candidate key.

Final Answer

Functional Dependencies:

CustomerID -> CustomerName, PhoneNo

PlanID -> PlanName, Amount

PhoneNo -> CustomerID

Candidate Keys:

CustomerID

PhoneNo

Primary Key:

CustomerID

Q8. A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.

Answer

Given:

PRODUCT(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

Assume the following dependencies:

PID -> PName, SupplierID, Category, Price, Warehouse

SupplierID -> SupplierName

Step 1: Check 1NF

All attributes contain atomic values.

Therefore:

PRODUCT is in 1NF.

Step 2: Check 2NF

The candidate key is:

PID

Since it is a single-attribute key, partial dependency is not possible.

Therefore:

PRODUCT is in 2NF.

Step 3: Check 3NF

We have:

PID -> SupplierID

SupplierID -> SupplierName

Therefore:

PID -> SupplierName

This is a transitive dependency.

So the original relation is not in 3NF.

Step 4: Decompose

Create PRODUCT:

PRODUCT(PID, PName, SupplierID, Category, Price, Warehouse)

Create SUPPLIER:

SUPPLIER(SupplierID, SupplierName)

Step 5: BCNF Check

In PRODUCT:

PID -> PName, SupplierID, Category, Price, Warehouse

PID is the key.

In SUPPLIER:

SupplierID -> SupplierName

SupplierID is the key.

Therefore, both relations satisfy BCNF under the assumed dependencies.

Final Normalized Design

PRODUCT(PID, PName, SupplierID, Category, Price, Warehouse)

SUPPLIER(SupplierID, SupplierName)

Result

The design removes the transitive dependency and reduces supplier-name redundancy.

Q9. Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.

Answer

Consider:

SCHOOL(

StudentID,

StudentName,

ClassID,

ClassName,

TeacherID,

TeacherName,

SubjectID,

SubjectName,

Marks

)

Step 1: Functional Dependencies

Assume:

StudentID -> StudentName, ClassID

ClassID -> ClassName, TeacherID

TeacherID -> TeacherName

SubjectID -> SubjectName

(StudentID, SubjectID) -> Marks

Step 2: Identify Problems

There are transitive dependencies:

StudentID -> ClassID

ClassID -> ClassName, TeacherID

Therefore:

StudentID -> ClassName, TeacherID

Also:

TeacherID -> TeacherName

Therefore teacher information is indirectly dependent on StudentID.

This creates redundancy.

Step 3: Identify Anomalies

Update anomaly: Teacher or class information may need to be updated in many student records.

Insertion anomaly: A new teacher or subject may not be stored without student information.

Deletion anomaly: Removing the last student in a class may remove class information.

Step 4: Decompose

Create STUDENT:

STUDENT(StudentID, StudentName, ClassID)

Create CLASS:

CLASS(ClassID, ClassName, TeacherID)

Create TEACHER:

TEACHER(TeacherID, TeacherName)

Create SUBJECT:

SUBJECT(SubjectID, SubjectName)

Create MARKS:

MARKS(StudentID, SubjectID, Marks)

Final Design

STUDENT(StudentID, StudentName, ClassID)

CLASS(ClassID, ClassName, TeacherID)

TEACHER(TeacherID, TeacherName)

SUBJECT(SubjectID, SubjectName)

MARKS(StudentID, SubjectID, Marks)

BCNF Justification

In each relation, the determinant is a key:

StudentID -> StudentName, ClassID

ClassID -> ClassName, TeacherID

TeacherID -> TeacherName

SubjectID -> SubjectName

(StudentID, SubjectID) -> Marks

Therefore, under the stated assumptions, each relation satisfies BCNF.

Q10. Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.

Answer

Consider:

STREAMING(MovieID, ActorID, PlatformID)

Suppose a movie can be associated independently with:

- actors
- streaming platforms

and there are independent relationships between these entities.

Step 1: Identify Join Dependency

The relation may contain combinations such as:

MovieID	ActorID	PlatformID
M01	A01	P01
M01	A01	P02
M01	A02	P01
M01	A02	P02

This may create redundancy because independent relationships are stored as combinations.

The relation can be decomposed into:

MOVIE_ACTOR(MovieID, ActorID)

MOVIE_PLATFORM(MovieID, PlatformID)

ACTOR_PLATFORM(ActorID, PlatformID)

Step 2: Join Dependency

The original relation can be reconstructed by joining the three relations:

STREAMING =

MOVIE_ACTOR

JOIN MOVIE_PLATFORM

JOIN ACTOR_PLATFORM

This represents a join dependency.

Step 3: 5NF Decomposition

Final relations:

MOVIE_ACTOR(MovieID, ActorID)

MOVIE_PLATFORM(MovieID, PlatformID)

ACTOR_PLATFORM(ActorID, PlatformID)

Step 4: Lossless-Join Verification

To verify lossless join:

1. Decompose the original relation into the three relations.
2. Join the decomposed relations using their common attributes.
3. Compare the resulting tuples with the original relation.
4. If no spurious tuples are generated and the original tuples are recovered, the decomposition is lossless.

Therefore:

JOIN(MOVIE_ACTOR, MOVIE_PLATFORM, ACTOR_PLATFORM)

= Original STREAMING relation

Hence:

Result = Lossless-Join 5NF Decomposition

Final Answer

MOVIE_ACTOR(MovieID, ActorID)

MOVIE_PLATFORM(MovieID, PlatformID)

ACTOR_PLATFORM(ActorID, PlatformID)

This removes join dependency-related redundancy and gives a 5NF design.